
Lưu trữ cơ sở dữ liệu bằng sử dụng cấu trúc B+ Tree trên ổ đĩa

Phan Gia Lộc^{1,*}, Đặng Ngọc Sang^{1,*}

¹Trường Đại học Khoa học tự nhiên

*Đóng góp ngang nhau

Tóm tắt nội dung

Bài báo cáo này được chúng tôi thực hiện nhằm hoàn thành đồ án môn học Cấu trúc dữ liệu và giải thuật. Cụ thể, tôi sẽ trình bày quá trình đánh giá hiệu năng của một hệ quản trị lưu trữ cơ sở dữ liệu dựa trên cấu trúc B+ Tree trên ổ đĩa, từ đó kết hợp với các kiến thức nâng cao, chuyên sâu hơn về mặt phần mềm, phần cứng máy tính và các kiến thức liên quan đến cơ sở, cấu trúc dữ liệu để hướng đến mục tiêu là không chỉ xác thực tính ưu việt về mặt lý thuyết của cấu trúc B+ Tree so với B-Tree truyền thống, mà còn cung cấp một góc nhìn thực tiễn dưới góc độ của một kỹ sư hệ thống. Đồng thời làm rõ mối quan hệ mật thiết giữa thuật toán, cấu trúc dữ liệu và giới hạn vật lý của thiết bị phần cứng, khẳng định lý do B+ Tree trở thành tiêu chuẩn vàng cho các hệ thống lưu trữ dữ liệu quy mô lớn hiện nay.

Ngày: Tháng 7, năm 2026

Liên hệ: 2512000638@student.hcmus.edu.vn và 2512000706@student.hcmus.edu.vn

Mã nguồn: https://github.com/LeatuyrBertyk/CSC10004-BPlus_Tree

1 Bối cảnh, mục đích và phạm vi đồ án

1.1 Bối cảnh thực tế

Trong kiến trúc của các hệ quản trị cơ sở dữ liệu quan hệ hiện đại (như MySQL hay PostgreSQL), khối lượng dữ liệu thực tế thường xuyên vượt quá sức chứa của bộ nhớ RAM. Để giải quyết vấn đề này, dữ liệu bắt buộc phải được lưu trữ trực tiếp trên bộ nhớ thứ cấp (ổ cứng HDD hoặc SSD) dưới dạng các khối dữ liệu có kích thước cố định được gọi là các Trang (Pages), thông thường với kích thước chuẩn là 4KB.

Tuy nhiên, sự chênh lệch về giới hạn vật lý khiến tốc độ truy xuất ổ đĩa chậm hơn hàng vạn lần so với RAM. Khi hệ thống phải xử lý lượng dữ liệu khổng lồ, chi phí cho các thao tác đọc/ghi đĩa ngay lập tức trở thành nút thắt cổ chai, kìm hãm toàn bộ hiệu năng. Vấn đề này càng bộc lộ rõ thông qua các thao tác phổ biến nhất trên cơ sở dữ liệu, điển hình là truy vấn theo khoảng giá trị (Range Query). Mặc dù lý thuyết nền tảng thường đề cập đến B-Tree, nhưng thực tiễn vận hành công nghiệp cho thấy 99% các hệ quản trị cơ sở dữ liệu đều lựa chọn kiến trúc B+ Tree làm cấu trúc cốt lõi.

1.2 Mục đích của báo cáo

Đồ án đưa ra nhằm giải thích về mặt thực nghiệm lý do tại sao cấu trúc B+ Tree lại giữ vị trí độc tôn trong ngành cơ sở dữ liệu. Bằng việc đóng vai trò như một Kỹ sư hệ thống, đồ án yêu cầu tiến hành xây dựng từ đầu một hệ thống lưu trữ dữ liệu hoàn chỉnh, sử dụng cấu trúc B+ Tree tương tác trực tiếp với bộ nhớ đĩa.

Thông qua các phương pháp đo lường hiệu năng, báo cáo sẽ cung cấp số liệu thực tế khi so sánh hai phương thức truy vấn kiểu B-Tree và B+ Tree trong cả truy vấn giá trị và truy vấn khoảng giá trị. Từ những dữ liệu thu thập được, báo cáo hướng tới việc phân tích sâu sắc bốn chủ đề nâng cao:

- Đánh giá hiệu năng truy vấn khoảng giá trị và giải quyết nút thắt quá trình đọc/ghi trên ổ đĩa.
- Tối ưu hóa cấu trúc nút và bài toán kiểm soát chiều cao của B+ Tree.
- Tương tác giữa giới hạn vi xử lý và giới hạn ổ đĩa.
- Hiện tượng phân mảnh dữ liệu và các liên hệ với thực tế hệ thống.

1.3 Phạm vi và yêu cầu kỹ thuật

Để đảm bảo mô hình phản ánh chính xác cơ chế hoạt động của phần cứng và hệ điều hành, hệ thống thực nghiệm được xây dựng tuân thủ các ràng buộc kỹ thuật sau:

Cấu trúc vật lý. Các bản ghi dữ liệu (Records) được thiết kế với kích thước cố định 64 byte. Các nút trên cây sử dụng từ khóa **union** trong C/C++ nhằm ép kích thước vật lý khớp chính xác với giới hạn 4096 byte (4KB) của một Trang (Pages) tiêu chuẩn. Điều này cho phép một nút Lá chứa tối đa 60 bản ghi, trong khi nút nội có sức chứa lên tới 510 khóa.

Quản lý bộ nhớ. Hệ thống hoạt động dưới ràng buộc dung lượng RAM tối đa chỉ 16MB. Tuyệt đối nghiêm cấm việc nạp toàn bộ cấu trúc cây vào bộ nhớ chính; thay vào đó, các nút chỉ được nạp/ghi đơn lẻ từ đĩa thông qua các cơ chế đọc/ghi tiêu chuẩn là **fread** và **fwrite**.

Liên kết dữ liệu. Toàn bộ con trỏ động trong C++ bị loại bỏ, thay vào đó, các nút trên cây liên kết với nhau hoàn toàn thông qua các chỉ số độ dời byte vật lý trên tệp nhị phân.

Phương thức tìm kiếm. Thuật toán đánh giá hiệu năng được áp dụng để kiểm tra độ phức tạp thuật toán bằng cách tích hợp cả tìm kiếm tuyến tính (Linear Search) và tìm kiếm Nhị phân (Binary Search) bên trong quá trình duyệt nút.

2 Đánh giá hiệu năng thực nghiệm

2.1 Môi trường thực thi và phương pháp đo lường

Để đảm bảo tính minh bạch của các kết quả thực nghiệm, toàn bộ quá trình biên dịch và đo lường được thực hiện trên một máy tính duy nhất là máy tính cá nhân của chúng tôi với mẫu máy Dell Inspiron 5458; vi xử lý Intel(R) Core(TM) i5-5250U hoạt động ở xung nhịp cơ bản 1.60GHz; bộ nhớ chính (RAM) có tổng dung lượng 12GB RAM chuẩn DDR3, tốc độ 1600 MT/s, được thiết lập chạy ở chế độ kênh đôi với hai thanh nhớ SODIMM (8GB của Samsung và 4GB của Micron); bộ nhớ lưu trữ là ổ cứng SSD ADATA SU650 với tổng dung lượng khoảng 480GB; và hệ điều hành là Ubuntu 25.10 (x86-64), sử dụng lõi Linux Kernel phiên bản 6.17.0-35-generic. Việc sử dụng SSD thay vì HDD cơ học đóng vai trò quan trọng trong việc giảm thiểu độ trễ vật lý khi hệ thống thực hiện các thao tác đọc/ghi dữ liệu.

Yêu cầu của quy trình đánh giá hiệu năng là chính xác đến mức độ nano giây, nhưng trên các hệ điều hành hiện đại thường xuyên bị nhiễu loạn bởi các yếu tố ngầm như các tiến trình chạy ngầm, và cơ chế đệm trang của hệ điều hành (OS Page Cache). Nhằm giảm thiểu tối đa các sai số vật lý này và thu thập được tốc độ thuần túy của thuật toán, chiến lược đo lường được thiết kế qua ba bước:

Khởi động. Tiến hành khởi chạy tệp thực thi lần đầu tiên sau khi tệp tin cơ sở dữ liệu nhị phân được tạo. Kết quả đo lường ở lần chạy này bị loại bỏ hoàn toàn do độ trễ khổng lồ từ việc đọc dữ liệu từ SSD lên RAM. Thao tác này nhằm ép hệ điều hành nạp sẵn các dữ liệu cần thiết vào vùng OS Page Cache.

Lấy mẫu. Khi dữ liệu đã nằm trên bộ nhớ đệm, hệ thống tiếp tục chạy tự động thao tác đánh giá 10 lần đọc lặp. Trong mỗi lần chạy, thuật toán phải thực hiện chính xác 100 truy vấn ngẫu nhiên cho từng kích thước dữ liệu N .

Thống nhất số liệu. Số lần đọc đĩa là một hằng số tuyệt đối, do đó thông số này được giữ nguyên không đổi. Tuy nhiên, đối với thời gian thực thi, báo cáo áp dụng phương pháp trích xuất giá trị thấp nhất trong 10 lần đo. Giá trị nhỏ nhất đại diện cho khoảnh khắc vi xử lý ít bị hệ điều hành can thiệp nhất, phản ánh trung thực nhất năng lực giới hạn của từng cấu trúc cây.

2.2 Tổng hợp kết quả quá trình thực nghiệm

Mục tiêu của thực nghiệm là đánh giá hiệu năng của hệ thống khi quy mô dữ liệu (N) tăng dần từ 1.000 lên đến mức cực đại 10.000.000 bản ghi. Tại mỗi mốc dữ liệu N , hệ thống sẽ thực hiện đánh giá các biến thể thuật toán, gồm B-Tree Style Linear Search, B-Tree Style Binary Search, B+ Style Tree Linear Search và B+ Style Tree Binary Search, thông qua hai tiêu chí đo lường cốt lõi: **thời gian thực thi trung bình** (tính bằng nano giây), đại diện cho tốc độ xử lý thuần túy của vi xử lý, bao gồm thời gian thực hiện các phép toán tìm kiếm nội bộ cộng với độ trễ tìm nạp bộ nhớ đệm; và **số lần đọc đĩa trung bình** (reads), đại diện cho nút thắt cổ chai quá trình đọc/ghi của hệ thống, thông số này đếm số lượng khối dữ liệu 4KB (một Trang - Pages) thực tế phải được nạp từ tệp nhị phân trên ổ cứng vào RAM (thông qua lệnh `fread`) để hoàn thành một truy vấn.

Kiểm nghiệm được chia thành hai thao tác cơ sở dữ liệu phổ biến nhất: **truy vấn điểm** (Point Query), tức tìm kiếm tính tồn tại của một ID cụ thể; và **truy vấn khoảng** (Range Query), quét và đếm số lượng bản ghi nằm trong một khoảng cho trước, kích thước khoảng quét `range_size` được thiết lập là 500 đơn vị với mốc dữ liệu nhỏ ($N < 2000$) và mở rộng lên 1000 đơn vị đối với các mốc dữ liệu lớn hơn.

Bảng 1 So sánh hiệu năng truy vấn điểm (Point Query).

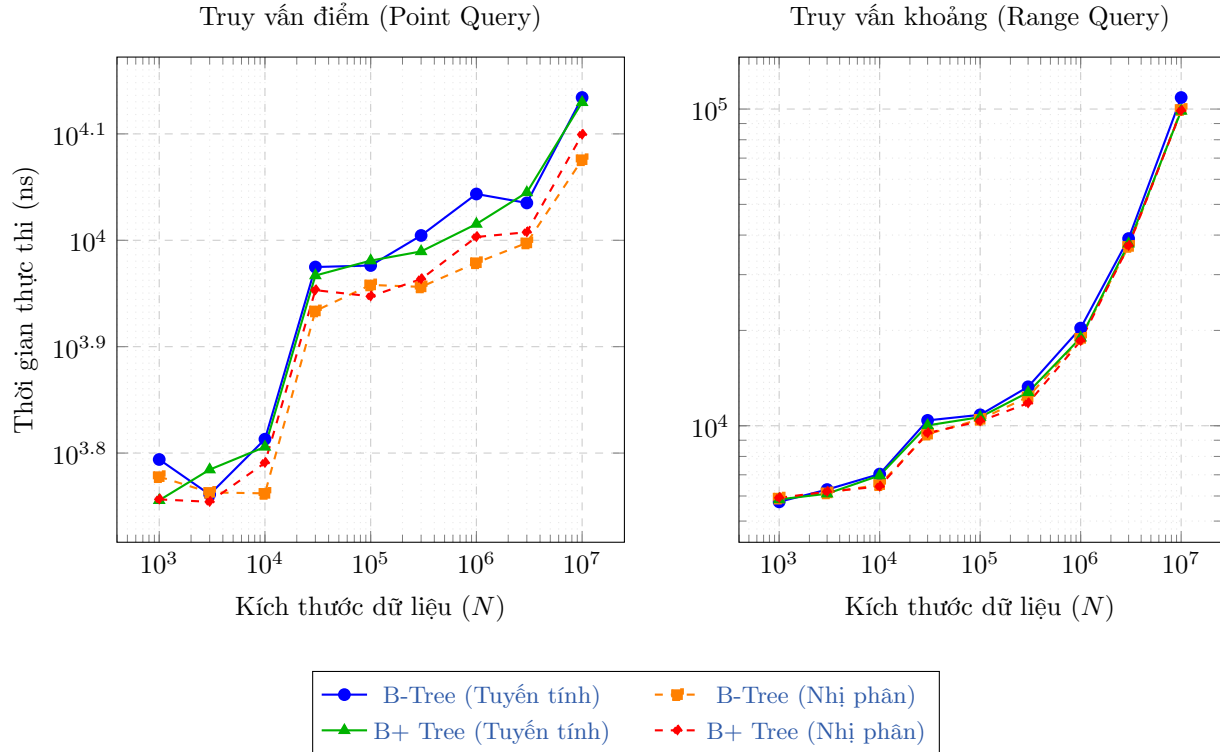
Kích thước dữ liệu (N)	Kiểu B-Tree (Tuyến tính)	Kiểu B-Tree (Nhị phân)	Kiểu B+ Tree (Tuyến tính)	Kiểu B+ Tree (Nhị phân)
1,000	6226 ns 1 reads	5999 ns 1 reads	5700 ns 2 reads	5711 ns 2 reads
3,000	5772 ns 1 reads	5798 ns 1 reads	6091 ns 2 reads	5681 ns 2 reads
10,000	6506 ns 1 reads	5786 ns 1 reads	6398 ns 2 reads	6181 ns 2 reads
30,000	9443 ns 2 reads	8587 ns 2 reads	9271 ns 3 reads	8982 ns 3 reads
100,000	9474 ns 2 reads	9084 ns 2 reads	9575 ns 3 reads	8863 ns 3 reads
300,000	10110 ns 2 reads	9046 ns 2 reads	9767 ns 3 reads	9199 ns 3 reads
1,000,000	11060 ns 2 reads	9531 ns 2 reads	10364 ns 3 reads	10077 ns 3 reads
3,000,000	10848 ns 2 reads	9949 ns 2 reads	11097 ns 3 reads	10182 ns 3 reads
10,000,000	13629 ns 3 reads	11911 ns 3 reads	13487 ns 4 reads	12577 ns 4 reads

Bảng 2 So sánh hiệu năng truy vấn khoảng (Range Query).

Kích thước dữ liệu (N)	Kiểu B-Tree (Tuyến tính)	Kiểu B-Tree (Nhị phân)	Kiểu B+ Tree (Tuyến tính)	Kiểu B+ Tree (Nhị phân)
1,000	5750 ns 2 reads	5909 ns 2 reads	5853 ns 2 reads	5921 ns 2 reads
3,000	6288 ns 2 reads	6137 ns 2 reads	6101 ns 2 reads	6190 ns 2 reads
10,000	7038 ns 2 reads	6519 ns 2 reads	6966 ns 2 reads	6430 ns 2 reads
30,000	10397 ns 3 reads	9410 ns 3 reads	10025 ns 3 reads	9485 ns 3 reads
100,000	10825 ns 3 reads	10481 ns 3 reads	10647 ns 3 reads	10335 ns 3 reads
300,000	13272 ns 3 reads	12329 ns 3 reads	12746 ns 3 reads	11782 ns 3 reads
1,000,000	20339 ns 5 reads	18877 ns 5 reads	18967 ns 5 reads	18541 ns 5 reads
3,000,000	39006 ns 10 reads	36841 ns 10 reads	37564 ns 10 reads	37010 ns 10 reads
10,000,000	108732 ns 28 reads	99669 ns 28 reads	98640 ns 27 reads	98768 ns 27 reads

Hai bảng số liệu đã cung cấp chi tiết về các thông số vật lý, nhưng để phân tích sâu hơn về xu hướng biến thiên của hệ thống khi khối lượng dữ liệu bùng nổ khi N càng lớn, các kết quả thực nghiệm cần được trực quan hóa. Do dải dữ liệu thử nghiệm (N) trải dài trên một biên độ rất lớn, việc sử dụng hệ tọa độ tuyến tính thông thường sẽ làm không thể hiện rõ sự khác biệt ở các mốc nhỏ. Do đó, cả hai trục tọa độ đều được áp dụng tỷ lệ logarit cơ số 10, nhằm trực quan hóa sự khác biệt trên.

Từ hai biểu đồ trên, chúng ta có thể rút ra hai quan sát sơ bộ như sau:



Hình 1 So sánh hiệu năng truy vấn điểm và truy vấn khoảng theo kích thước dữ liệu (N).

Sự thống trị của B-Tree trong thao tác truy vấn điểm. Quan sát biểu đồ 1, các đường biểu diễn hiệu năng của B-Tree (đặc biệt là đường B-Tree Binary) gần như luôn nằm dưới cùng, thể hiện thời gian thực thi nhanh nhất ở mọi mức dữ liệu. Điều này minh chứng cho lợi thế của cơ chế thoát sớm (early exit) của cấu trúc B-Tree, tức khi ID cần tìm vô tình nằm ngay tại các nút nội ở tầng cao, thuật toán có thể trả về kết quả ngay lập tức mà không cần phải duyệt xuống tận tầng đáy như giới hạn bắt buộc của cấu trúc B+ Tree.

Sức mạnh ẩn của B+ Tree trong truy vấn khoảng (Range Query). Quan sát biểu đồ 1, mặc dù B+ Tree có tổng số lần đọc đĩa ít hơn ở mức $N = 10.000.000$, nhưng thời gian thực thi lại chưa tạo ra khoảng cách áp đảo so với kiểu B-Tree. Hiện tượng này xảy ra do dải truy vấn trong bài kiểm tra chưa đủ lớn để đánh bại bộ đệm của hệ điều hành (OS Page Cache). Khi dữ liệu vẫn còn nằm gọn trong RAM, tốc độ nhảy giữa các lần đệ quy của B-Tree chưa bộc lộ độ trễ. Sức mạnh thực sự của liên kết tại các nút lá của B+ Tree sẽ chỉ phát huy thật sự khi dải khoảng giá trị `range_size` được mở rộng lên hàng triệu đơn vị, khi quá tải RAM và ép hệ thống phải đọc vật lý từ ổ đĩa vật lý.

3 Hiệu năng truy vấn khoảng và nút thắt cổ chai đọc/ghi trên ổ đĩa

Như đã đề cập trong phần nhận xét của biểu đồ 1, B-Tree luôn thống trị trong thao tác truy vấn khoảng, tuy nhiên B+ Tree lại mang sức mạnh tiềm ẩn trong thao tác truy vấn khoảng. Dù điểm mạnh ấy chưa được thể hiện một cách vượt bậc trong chạy thử nghiệm, nhưng với những bộ dữ liệu khổng lồ hơn, tiềm năng ấy sẽ được phát huy vượt bậc. Để giải thích cho nhận định trên, chúng tôi xin trình bày cách tiếp cận sau.

Trong cấu trúc B-Tree truyền thống, dữ liệu được lưu trữ phân tán ở cả các nút nội lẫn nút lá. Khi thực hiện truy vấn khoảng, yêu cầu thực hiện quét qua một loạt các bản ghi theo thứ tự, thuật toán bắt buộc phải duyệt cây theo chiều sâu. Việc duyệt này đòi hỏi hệ thống phải duy trì một ngăn xếp (stack) trong bộ nhớ chính để lưu trữ các nút dọc theo đường dẫn từ gốc nhằm tránh việc phải đọc lại chúng nhiều lần.

Tuy nhiên, để tìm các khóa tiếp theo trong một khoảng giá trị lớn, thuật toán liên tục phải nhảy từ nút lá

này ngược lên nút cha rồi đi xuống út lá khác. Các bước nhảy này buộc đầu đọc của ổ đĩa phải liên tục di chuyển đến các vị trí vật lý khác nhau, sinh ra hàng loạt thao tác đọc đĩa ngẫu nhiên. Chi phí cho các truy cập ngẫu nhiên này cực kỳ đắt đỏ vì mỗi lần đọc đều phải gánh chịu toàn bộ thời gian dành cho việc tìm kiếm công với độ trễ.

Để giải quyết vấn đề trên, B+ Tree được sinh ra để tối ưu hóa thông qua đọc/ghi tuần tự. B+ Tree giải quyết triệt để nút thắt cổ chai đọc/ghi ngẫu nhiên của B-Tree bằng cách tái cấu trúc lại việc lưu trữ. Trong B+ Tree, toàn bộ dữ liệu thực tế đều nằm ở các nút lá, trong khi các nút ở tầng cao hơn chỉ đóng vai trò như một bản đồ chỉ mục để định tuyến việc tìm kiếm. Cải tiến mang tính cốt lõi của B+ Tree là việc các nút lá được liên kết với nhau từ trái sang phải, tạo thành một danh sách liên kết được gọi là *tập tuần tự*. Trong cài đặt thực tế, danh sách này được duy trì bằng một con trỏ (cụ thể là `next_leaf_offset`). Nhờ có *tập tuần tự*, thao tác truy vấn khoảng trở nên vô cùng đơn giản, thuật toán chỉ cần tìm kiếm điểm để chạm đến phần tử bắt đầu, sau đó chỉ việc duyệt dọc theo danh sách liên kết của các nút lá để lấy toàn bộ dữ liệu tiếp theo. Về mặt phần cứng, cấu trúc này cho phép hệ thống đọc các nút lá một cách tuần tự. Việc đọc khối dữ liệu tuần tự có thể nhanh hơn gấp 10 lần so với đọc ngẫu nhiên, bởi vì nó phân bổ chi phí dành cho thời gian tìm kiếm và độ trễ trên nhiều trang đĩa liên tiếp [2].

Cụ thể, kết quả chạy thực nghiệm với dữ liệu cấu hình tối đa ($N = 10,000,000$, sử dụng thuật toán tìm kiếm nhị phân) cung cấp một góc nhìn thực tế về sự khác biệt giữa hai cấu trúc. Theo bảng dữ liệu, đối với truy vấn khoảng với kích thước `range_size = 1000`, B-Tree Style tốn 28 lần đọc đĩa (reads) và mất 99,669 ns, trong khi B+ Tree Style tốn 27 lần đọc đĩa (reads) và mất 98,768 ns. Tuy nhiên, so với lý thuyết, kết quả thực nghiệm này chỉ cho thấy B+ Tree giảm được 1 lần đọc đĩa và nhanh hơn khoảng khoảng 1000 nano giây so với B-Tree. Sự chênh lệch này chưa thể hiện được sự khác biệt khổng lồ như kỳ vọng vì những giới hạn cụ thể của môi trường thực nghiệm, cụ thể:

Cơ chế OS Page Cache. Hệ điều hành tự động đệm các tệp đệm vào RAM thông qua OS Page Cache. Vì thử nghiệm chỉ chạy 100 truy vấn ngẫu nhiên trên cùng một tệp dữ liệu, phần lớn các nút cấp cao của cây đã nằm sẵn trong Cache. Thao tác đọc đĩa lúc này thực chất là truy xuất từ RAM, làm lu mờ sự đắt đỏ của độ trễ vật lý, vốn là điểm yếu chí mạng của đọc/ghi ngẫu nhiên.

Kích thước truy vấn `range_size` quá nhỏ. Với kích thước một trang đĩa là 4KB, một nút có thể chứa tối đa 60 bản ghi. Do đó, truy vấn khoảng với `range_size = 1000` chỉ kéo dài qua $1000/60 \approx 17$ nút lá. Việc duyệt ngắn xếp lên xuống qua 17 nút trên môi trường RAM tốn rất ít chu kỳ vi xử lý, không đủ để tạo ra khoảng cách lớn về nano giây so với việc duyệt danh sách liên kết của B+ Tree.

Dù môi trường mô phỏng chưa bộc lộ toàn bộ áp lực của đĩa vật lý do sự can thiệp của OS Cache và giới hạn của `range_size`, cấu trúc B+ Tree vẫn chứng minh được nguyên lý hoạt động ưu việt của nó. Sự tồn tại của trường `next_leaf_offset` là nhân tố quyết định giúp triệt tiêu hoàn toàn sự cần thiết của thuật toán đệ quy khi quét dữ liệu. Trong các cơ sở dữ liệu thực tế với lượng truy xuất hàng triệu bản ghi, cơ chế này giúp chuyển đổi hoàn toàn đọc/ghi ngẫu nhiên sang đọc/ghi tuần tự, tối đa hóa thông lượng của thiết bị lưu trữ vật lý.

4 Chiều cao B+ Tree và tối ưu hóa cấu trúc nút

Trước khi đi vào phần này, ta cần nhắc lại lưu ý rằng theo thiết kế của hệ thống, bộ nhớ thứ cấp được chia thành các Trang (Pages) có kích thước cố định là 4KB nhằm mô phỏng hoàn hảo cách hệ điều hành và ổ cứng vật lý hoạt động. Mỗi bản ghi có kích thước cố định 64 byte (gồm 4 byte ID và 60 byte `payload`), do đó nút lá chứa tối đa 60 bản ghi, nút nội vì chỉ lưu trữ các khóa và con trỏ để định tuyến chứ không chứa dữ liệu thực, một nút nội có thể chứa tối đa 510 khóa.

4.1 Mối tương quan giữa truy vấn điểm và chiều cao cây

Trong các hệ thống quản trị cơ sở dữ liệu, chi phí đọc/ghi trên đĩa là nút thắt cổ chai lớn nhất. Đối với cấu trúc B+ Tree, thao tác truy vấn điểm yêu cầu thuật toán phải duyệt từ nút gốc đi xuống tận nút lá để tìm kiếm bản ghi. Vì cơ chế bộ nhớ ngoài giới hạn mỗi nút là một trang (Page), mỗi lần thuật toán đọc một cấp của cây sẽ tương đương với một lần truy xuất đĩa cứng. Do đó, số lần đọc đĩa trong thao tác truy vấn điểm

phản ánh trực tiếp chiều cao của cây B+ Tree. Phân tích sự thay đổi của số lần đọc đĩa khi kích thước dữ liệu (N) tăng lên sẽ cho phép chúng ta đánh giá mô hình tăng trưởng của chiều cao cây và hiệu quả của cấu trúc lưu trữ.

Để tìm hiểu cụ thể và mang tính học thuật hơn, ta cần xác định rõ chiều cao cây tăng theo hàm dạng tuyến tính, logarit hay dạng nào khác. Trước tiên, đối chiếu trực tiếp với dữ liệu thực nghiệm từ bảng 1, dữ liệu thực tế cho thấy số lần đọc đĩa của B+ Tree thay đổi khi quy mô dữ liệu N mở rộng, cụ thể: tại $N = 1,000$ đến $N = 10,000$, số lần đọc đĩa là 2; tại $N = 30,000$ đến $N = 3,000,000$, số lần đọc đĩa tăng lên 3; và tại $N = 10,000,000$, số lần đọc đĩa chỉ tăng lên 4. Khi này, ta có nhận xét rằng khối lượng dữ liệu N đã tăng lên gấp 10,000 lần (từ 1,000 lên 10,000,000 bản ghi), nhưng chiều cao của cây, cũng tức là số lần đọc đĩa chỉ tăng thêm 2 (từ 2 lên 4). Tốc độ tăng này rất chậm, nên ta có thể đưa ra dự đoán rằng: *chiều cao của B+ Tree tăng theo mô hình hàm logarit*. Nếu chứng minh được, ta sẽ khẳng định được B+ Tree đạt được một cấu trúc rất nông và mở rộng theo chiều ngang thay vì theo chiều dọc.

Bây giờ, chúng tôi sẽ đề xuất cách chứng minh dự đoán trên bằng các kiến thức toán học.

Nhận xét 1. B+ Tree xây dựng từ N bản ghi có chiều cao tăng theo hàm logarit phụ thuộc vào N .

Chứng minh. Trước tiên, ta đặt $C_L = 60$ là số bản ghi lớn nhất mà một nút lá có thể chứa, nên trong trường hợp ít dữ liệu nhất thì mỗi nút lá chứa ít nhất $C_L/2$ bản ghi. Đặt $C_I = 510$ là số khóa lớn nhất mà một nút nội có thể chứa, nên trong trường hợp ít dữ liệu nhất thì mỗi nút nội chứa ít nhất $C_I/2$ khóa, đồng nghĩa sẽ có $C_I/2 + 1$ con trở đến các nút con.

Gọi h là chiều cao của B+ Tree có N bản ghi. Ta sẽ chứng minh bất đẳng thức:

$$\log_{C_I+1} \left(\frac{2N}{C_L} \right) + 1 \leq h \leq \log_{C_I/2+1} \left(\frac{N}{C_L} \right) + 2. \quad (1)$$

Xét B là B+ Tree được xây dựng từ N bản ghi và có chiều cao h . Trong trường hợp mà $B \equiv B_1$ vẫn giữ chiều cao h nhưng có ít bản ghi nhất, ở tầng 1, B_1 có 1 nút gốc; ở tầng 2, B_1 có 2 nút (là 2 nút con của nút gốc); ở tầng 3, B_1 có $2 \cdot \left(\frac{C_I}{2} + 1\right)$ nút (do mỗi nút ở tầng 2 có ít nhất $C_I/2 + 1$ con trở đến các nút con); ở tầng 4, B_1 có $2 \cdot \left(\frac{C_I}{2} + 1\right)^2$ nút; tương tự vậy, ta có ở tầng h (tức ở tầng chứa các nút lá), B_1 có $2 \cdot \left(\frac{C_I}{2} + 1\right)^{h-2}$ nút. Do mỗi nút lá chứa ít nhất $C_L/2$ bản ghi, nên ta có bất đẳng thức

$$N \geq 2 \cdot \left(\frac{C_I}{2} + 1 \right)^{h-2} \cdot \frac{C_L}{2} = C_L \cdot \left(\frac{C_I}{2} + 1 \right)^{h-2}.$$

Từ đây, ta dễ dàng biến đổi và thu được

$$h \leq \log_{C_I/2+1} \left(\frac{N}{C_L} \right) + 2. \quad (2)$$

Còn trong trường hợp $B \equiv B_2$ vẫn giữ chiều cao h nhưng có nhiều bản ghi nhất, ở tầng 1, B_2 có 1 nút gốc; ở tầng 2, B_2 có $C_I + 1$ nút (do nút gốc chứa đầy đủ C_I khóa; ở tầng 3, B_2 có $(C_I + 1)^2$ nút; tương tự vậy, ở tầng h , B_2 có $(C_I + 1)^{h-1}$ nút. Do mỗi nút lá chứa nhiều nhất C_L bản ghi, nên ta có bất đẳng thức

$$N \leq (C_I + 1)^{h-1} \cdot C_L.$$

Từ đó, ta dễ dàng suy ra

$$h \geq \log_{C_I+1} \left(\frac{N}{C_L} \right) + 1. \quad (3)$$

Từ hai bất đẳng thức (2) và (3), ta suy ra bất đẳng thức (1) được chứng minh. *Lưu ý: bất đẳng thức này phù hợp với kết quả ở bảng 1, ví dụ với $N = 100,000$ thì $\log_{511}(2 \cdot 100,000/60) + 1 \leq h \leq \log_{256}(100,000/60) + 2$, tức $2, 3 \leq h \leq 3, 3$, tương đương $h = 3$, phù hợp với 3 lần đọc ở kiểu B+ Tree tuyến tính và nhị phân (chỉ so sánh với B+ Tree do B Tree có cơ chế thoát sớm nên bị thiếu một lần đọc ở nút lá).*

Từ bất đẳng thức (1), ta suy ra rằng $h \approx \mathcal{O}(\log N)$. Điều này chứng tỏ dự đoán ở trên là đúng. $\square$

4.2 Tối ưu hóa cấu trúc nút và sức mạnh của hệ số rẽ nhánh

Theo kết quả ở phần trên, nguyên nhân cốt lõi giúp hàm logarit của chiều cao B+ Tree có độ dốc cực thấp nằm ở việc tối ưu hóa hệ số rẽ nhánh, ký hiệu là d , đặc biệt $d_{\max} = C_I + 1 = \text{MAX_INTERNAL_KEYS} + 1 = 511$. Với hệ số rẽ nhánh khổng lồ như vậy, không gian chứa của cây mở rộng theo cấp số nhân với số mũ là chiều cao h , với $h \approx \mathcal{O}(\log N)$. Đồng thời, theo [4], trong điều kiện thực hiện thao tác thêm dữ liệu ngẫu nhiên, các nút sẽ chỉ đầy khoảng $\ln 2 \approx 69\%$ sức chứa của mình. Do vậy, ngay cả khi tính đến hiện tượng phân mảnh, một cây B+ Tree với chiều cao $h = 3$ hay $h = 4$ hoàn toàn đủ dung lượng vật lý để quản lý hàng chục triệu đến hàng trăm triệu bản ghi. Thực nghiệm tại $N = 10,000,000$ với chỉ 4 lần đọc đĩa, là bằng chứng tuyệt đối cho năng lực này.

Nếu so sánh với các cấu trúc dữ liệu trên bộ nhớ chính (RAM) truyền thống như cây nhị phân tìm kiếm hay cây AVL, ta sẽ thấy sự khác biệt mang tính sống còn đối với các hệ quản trị cơ sở dữ liệu. Trong cây nhị phân, hệ số rẽ nhánh luôn bị cố định ở $d = 2$, nên nếu lưu trữ 10 triệu bản ghi, chiều cao của cây nhị phân cân bằng sẽ xấp xỉ $\log_2(10,000,000) \approx 24$. Nếu mang cấu trúc này xuống ổ đĩa vật lý, mỗi thao tác truy vấn điểm sẽ đòi hỏi 24 lần đọc/ghi ngẫu nhiên. Do tốc độ đọc đĩa chậm hơn RAM hàng chục ngàn lần, đây là một thảm họa đối với hiệu năng hệ thống.

Do vậy, việc thiết kế nút vừa vặn với kích thước của một Trang (Page) 4KB để ép hệ số rẽ nhánh d lên mức tối đa là một chiến lược thiết kế có chủ đích rất hay dành cho hệ thống. Một hệ số rẽ nhánh lớn giữ cho cây B+ Tree luôn ở trạng thái nông, đảm bảo rằng dù dữ liệu có phình to đến kích thước Big Data (N lên tới hàng tỷ), số lần đọc đĩa cứng vẫn luôn duy trì ở mức hằng số rất nhỏ. Đây chính là lý do B+ Tree trở thành một tiêu chuẩn cho các hệ quản trị cơ sở dữ liệu trong việc duy trì tốc độ các thao tác với chi phí đọc/ghi cực thấp.

5 Sự tương tác giữa giới hạn thời gian vi xử lý và giới hạn quá trình đọc/ghi

Trong thiết kế của các hệ quản trị cơ sở dữ liệu, chi phí truy vấn thường bị chi phối bởi hai giới hạn chính, là thời gian xử lý của vi xử lý và độ trễ của quá trình truy xuất bộ nhớ ngoài. Để đánh giá sự tương tác giữa hai yếu tố này, chúng ta xem xét sự khác biệt khi áp dụng hai thuật toán tìm kiếm nội bộ khác nhau bên trong một nút là tìm kiếm tuyến tính và tìm kiếm nhị phân.

Dựa vào bảng 1, tại quy mô dữ liệu lớn nhất $N = 10,000,000$, ta quan sát được nếu sử dụng thuật toán tìm kiếm tuyến tính, hệ thống tiêu tốn 13,487 ns thời gian chạy và 4 lần đọc đĩa; trong khi các thông số khi sử dụng thuật toán tìm kiếm nhị phân là 12,577 ns thời gian chạy và 4 lần đọc đĩa. Từ đó ta có nhận xét rằng yếu tố thay đổi rõ rệt nhất giữa hai chiến lược tìm kiếm là thời gian vi xử lý thực hiện (giảm khoảng gần 1,000 ns), trong khi số lần đọc đĩa hoàn toàn không thay đổi (cố định ở mức 4).

5.1 Sự tách biệt giữa giới hạn thời gian vi xử lý và giới hạn quá trình đọc/ghi

Sự cố định của số lần đọc đĩa và sự sụt giảm của thời gian vi xử lý thực hiện phản ánh chính xác kiến trúc phân tầng của hệ thống lưu trữ, cụ thể:

Giới hạn quá trình đọc/ghi. Số lần đọc đĩa được quyết định hoàn toàn bởi chiều cao của cây B+ Tree, tức là số cấp nút cần phải đi qua để từ gốc xuống đến lá. Tại $N = 10,000,000$, cây có chiều cao là 4, do đó thuật toán luôn yêu cầu 4 thao tác đọc/ghi để tải 4 khối dữ liệu (mỗi khối 4KB) từ ổ cứng vật lý vào bộ nhớ RAM. Dù áp dụng bất kỳ thuật toán nào bên trong nút, thì toàn bộ 4KB dữ liệu vẫn phải được nạp vào RAM trong một lần đọc duy nhất. Vì vậy, số lần đọc đĩa hoàn toàn không bị ảnh hưởng bởi thuật toán tìm kiếm nội bộ.

Giới hạn thời gian vi xử lý. Sau khi Trang (Page) đã được nạp thành công vào bộ nhớ chính (RAM), thuật toán tìm kiếm mới bắt đầu được vi xử lý thực thi. Theo thiết kế, mỗi nút nội chứa tối đa 510 khóa. Với thuật toán tìm kiếm tuyến tính, vi xử lý phải thực hiện phép so sánh tuần tự, tốn trung bình khoảng $510/2 = 255$ vòng lặp để xử lý cho mỗi nút. Trong khi đó, với thuật toán tìm kiếm nhị phân, số phép so sánh tối đa ở mỗi nút chỉ là $\log_2(510) \approx 9$ vòng lặp xử lý; việc giảm số lượng phép toán từ 255 xuống 9 tại mỗi nút trên đường dẫn tìm kiếm đã làm giảm đáng kể chu kỳ máy, từ đó trực tiếp tối ưu hóa thời gian thực thi tổng thể mà kết quả thực nghiệm đã ghi nhận.

Sau phân tích trên, chắc hẳn không ít người sẽ thắc mắc: Trong bài báo cáo này, kích thước của mỗi nút được đặt ở mức 4KB, một con số khá nhỏ. Về mặt thiết kế hệ thống, nếu cấu hình kích thước Trang (Page) thực tế lớn hơn, ví dụ như 16KB hay 32KB, thuật toán tìm kiếm nội bộ sẽ tác động như thế nào đến vi xử lý? Phần sau đây sẽ giải đáp thắc mắc trên.

5.2 Tác động của kích thước Trang (Page) lớn lên hiệu năng vi xử lý

Sự bùng nổ của nút thất cổ chai vi xử lý. Nếu kích thước Page tăng lên 16KB hay 32KB, số lượng khóa chứa trong một nút nội sẽ tăng trưởng tuyến tính (có thể lên tới khoảng 2,000 đến 4,000 khóa mỗi nút). Lúc này, thời gian thực thi của tìm kiếm tuyến tính sẽ tăng vọt, đòi hỏi hàng ngàn chu kỳ xử lý chỉ để quét qua một nút, điều này sẽ khiến quá trình xử lý trên RAM trở thành một nút thất cổ chai thực sự. Ngược lại, tìm kiếm nhị phân tiếp tục chứng minh sự ưu việt tuyệt đối do bản chất của nó khi để tìm kiếm trong mảng 4,000 phần tử, nó chỉ mất tối đa $\log_2(4000) \approx 12$ phép so sánh (chỉ tăng thêm 3 phép so sánh so với mảng 510 phần tử). Do đó, khi kích thước Trang (Page) càng lớn, vai trò của tìm kiếm nhị phân càng mang tính sống còn đối với hiệu năng của hệ thống.

Ảnh hưởng của bộ đệm vi xử lý. Theo nghiên cứu của Rao và Ross trong bài báo "Making B+ Trees Cache Conscious in Main Memory"[3], khi dữ liệu trên RAM ngày càng lớn, độ trễ khi truy xuất từ RAM vào vi xử lý bắt đầu bộc lộ nhược điểm. Vi xử lý hiện đại lưu trữ dữ liệu tạm thời vào các Cache Line có kích thước rất nhỏ, thường từ 32 đến 128 byte; khi kích thước nút là 16KB hay 32KB, nó vượt xa sức chứa của một Cache Line. Nếu thực hiện tìm kiếm tuyến tính, vi xử lý sẽ liên tục phải tải các phần dữ liệu mới từ RAM vào Cache, gây ra hàng loạt hiện tượng Cache Misses đắt đỏ. Mặc dù tìm kiếm nhị phân giảm được số lần so sánh, nhưng do nó nhảy ngẫu nhiên giữa các nửa mảng của một nút 32KB, hiện tượng Cache Misses vẫn sẽ xảy ra ở những phép so sánh đầu tiên. Để giải quyết triệt để nút thất giới hạn vi xử lý trên các nút lớn, hệ thống cần tiến tới các thiết kế thân thiện với bộ đệm như CSB+-Tree, trong đó các nút con được nhóm lại và loại bỏ bớt con trỏ, cho phép số lượng khóa chứa trong một Cache Line được tối đa hóa, giúp đồng bộ hóa tốc độ xử lý của vi xử lý với độ trễ của RAM.

Do vậy, dữ liệu thực nghiệm đã chứng minh rõ sự phân ly giữa giới hạn quá trình đọc/ghi, được tối ưu bởi chiều cao cây B+ Tree; và giới hạn thời gian vi xử lý, được tối ưu bằng thuật toán nội bộ. Trong kỷ nguyên mà dung lượng RAM lớn cho phép sử dụng các Trang (Page) có kích thước khổng lồ, việc tối ưu hóa giới hạn thời gian vi xử lý thông qua tìm kiếm nhị phân và các kiến trúc thân thiện với Cache là bước đi bắt buộc để hệ quản trị cơ sở dữ liệu đạt được hiệu suất tối đa.

6 Hiện tượng phân mảnh dữ liệu và thực tế hệ thống

6.1 Bản chất của sự phân mảnh vật lý trong B+ Tree

Trong mô hình lưu trữ đĩa, B+ Tree dựa vào cấp phát khối bộ nhớ có kích thước cố định 4KB, khi hệ thống phải chèn hàng triệu bản ghi ngẫu nhiên, các nút lá sẽ liên tục đạt đến ngưỡng giới hạn và buộc phải thực hiện thao tác tách nút. Theo cơ chế tách nút, một nửa dữ liệu của nút hiện tại sẽ được di chuyển sang một nút mới, và nút mới này thường được cấp phát bằng cách nối thêm vào cuối tệp vật lý .dat. Hậu quả tất yếu của quá trình này là sự phá vỡ tính liên tục về mặt vật lý. Về mặt logic, hai nút lá chứa các khoảng giá trị liên tiếp nhau vẫn được kết nối hoàn hảo thông qua con trỏ `next_leaf_offset`; nhưng về mặt vật lý trên ổ cứng, nút lá thứ i có thể nằm ở offset 4096, trong khi nút lá thứ $i + 1$, vừa được sinh ra thông qua thao tác tách nút, lại nằm ở offset 40,000,000. Sự sai lệch giữa thứ tự logic và thứ tự vật lý này được gọi là hiện tượng phân mảnh dữ liệu, và nó phá hủy hoàn toàn chiến lược đọc tuần tự mà cấu trúc B+ Tree hướng tới.

6.2 Tác động của phân mảnh lên các thiết bị lưu trữ vật lý

Sự phân mảnh dữ liệu gây ra những hệ lụy suy giảm hiệu năng rất khác nhau tùy thuộc vào bản chất vật lý của thiết bị lưu trữ.

Đối với ổ đĩa cứng cơ học (HDD). HDD phụ thuộc vào chuyển động cơ học của đầu đọc/ghi và độ trễ quay của đĩa. Khi thực hiện truy vấn điểm trên một cây B+ Tree bị phân mảnh nặng, việc duyệt qua con trỏ

`next_leaf` buộc đầu đọc phải liên tục nhảy giữa các vùng cách xa nhau trên mặt đĩa, thao tác đọc tuần tự logic lúc này bị biến chất hoàn toàn thành đọc/ghi vật lý ngẫu nhiên. Và chi phí đọc/ghi ngẫu nhiên chậm hơn đọc/ghi tuần tự hàng chục đến hàng trăm lần, khiến hiệu năng truy vấn khoảng sụt giảm nghiêm trọng.

Đối với ổ cứng thể rắn (SSD). Khác với HDD, SSD lưu trữ dữ liệu trên các chip nhớ Flash và không có bộ phận chuyển động cơ học, nhờ đó, thao tác đọc ngẫu nhiên trên SSD gần như nhanh tương đương đọc tuần tự, giúp SSD được xem như miễn nhiệm với độ trễ truy vấn do phân mảnh đọc. Tuy nhiên, cấu trúc chip nhớ Flash lại có một điểm yếu chí mạng là đặc tính xóa trước khi ghi. Cụ thể, khi B+ Tree thực hiện các thao tác thêm nút ngẫu nhiên gây hiện tượng tách nút, nó tạo ra vô số thao tác ghi ngẫu nhiên với kích thước nhỏ 4KB rải rác. Theo nghiên cứu của Chen và cộng sự [1], khi SSD bị phân mảnh, các luồng ghi ngẫu nhiên và cập nhật tại chỗ khiến lớp dịch địa chỉ phải liên tục thực hiện quá trình thu gom rác, dời đổi khối và xóa dữ liệu. Điều này không chỉ làm giảm băng thông tổng thể một cách thảm hại mà còn bào mòn tuổi thọ vật lý của SSD.

6.3 Giải pháp thực tiễn từ các hệ quản trị cơ sở dữ liệu hiện đại

Để đối phó với nút thắt cổ chai do quá trình đọc/ghi ngẫu nhiên và sự phân mảnh, các hệ quản trị cơ sở dữ liệu thương mại như MySQL InnoDB, PostgreSQL, Oracle, và giới nghiên cứu đã áp dụng nhiều chiến lược từ bảo trì vận hành đến thiết kế lại kiến trúc, điển hình như:

Kỹ thuật bảo trì và bộ đệm. Kỹ thuật này áp dụng hai quy tắc cốt lõi là: **tái cấu trúc chỉ mục**, các quản trị viên cơ sở dữ liệu sẽ định kỳ chạy lệnh thiết lập lại cây, ví dụ: `OPTIMIZE TABLE` trong MySQL, để quét và ghi lại toàn bộ dữ liệu ra một tệp mới theo đúng thứ tự logic, giúp khôi phục hoàn toàn tính có tuần tự vật lý; và **sử dụng Buffer Pool**, các thao tác ghi không được đẩy thẳng xuống đĩa mà được giữ lại ở RAM, các bản ghi ngẫu nhiên được gom nhóm lại trong Buffer trước khi ghi xuống đĩa, làm giảm số lượng ghi ngẫu nhiên vật lý.

Cải tiến kiến trúc lưu trữ. Trong cấu trúc B+ Tree, như đã trình bày trong các phần trước, khi hệ thống thực hiện thao tác chèn và cập nhật với tần suất cao, các nút đáng lẽ phải nằm cạnh nhau về mặt logic lại nằm xa nhau về mặt vật lý trên ổ cứng. Hậu quả là làm giảm hiệu năng xuống hàng ngàn lần. Một giải pháp kiến trúc mang tính cách mạng đã được đề xuất bởi O’Neil và các cộng sự [2], đó là cấu trúc LSM-Tree. LSM-Tree ra đời dựa trên một triết lý trái ngược hoàn toàn là “Append-only”, tức không bao giờ ghi đè, chỉ ghi nối thêm. Thuật toán này chuyển đổi toàn bộ thao tác ghi ngẫu nhiên thành ghi tuần tự, ép ổ cứng phải hoạt động ở tốc độ tối đa của nó. Kiến trúc của LSM-Tree gồm 3 tầng cốt lõi:

- **Tầng RAM (MemTable - Memory Table).** Thay vì ghi xuống đĩa ngay lập tức, mọi lệnh chèn, cập nhật, xóa đều được ghi vào một cấu trúc dữ liệu nhỏ trên RAM (thường là cây đỏ đen hoặc danh sách nhảy - còn được gọi là đường cao tốc trong danh sách liên kết), làm cho tốc độ ghi lúc này nhanh bằng tốc độ của RAM.
- **Tầng Đĩa (SSTable - Sorted String Table).** Khi MemTable trên RAM bị đầy, hệ thống sẽ đóng băng nó lại và xả toàn bộ dữ liệu xuống ổ cứng thành một file gọi là SSTable. Vì dữ liệu trong MemTable đã được sắp xếp sẵn, quá trình xả hoàn toàn là đọc/ghi tuần tự, điều này khiến cách ghi không chỉ thích hợp với SSD mà còn rất ổn định khi sử dụng ổ cứng HDD. Và có lưu ý rằng, file SSTable này là bất biến, không bao giờ bị sửa đổi trong toàn bộ quá trình sử dụng hệ thống.
- **Cơ chế dọn dẹp (Compaction.)** Vì không sử dụng thao tác ghi đè, nên nếu một ID được cập nhật n lần, nó sẽ sinh ra n bản ghi nằm ở n file SSTable khác nhau, để giải quyết rắc rối này, hệ thống có một tiến trình chạy ngầm gọi là cơ chế dọn dẹp. Cơ chế này sẽ âm thầm đọc các file SSTable nhỏ, gộp chúng lại, xóa các dữ liệu cũ hay ít sử dụng gần đây, và ghi ra một file SSTable lớn hơn, sạch sẽ hơn ở các tầng sâu hơn.

Qua đó, ta có nhận xét rằng nguyên lý tối ưu cốt lõi của LSM-Tree là gom hàng vạn thao tác ghi ngẫu nhiên rời rạc thành một chuỗi ghi tuần tự lớn đổ xuống đĩa. Điều này giúp các nút trên đĩa luôn ở trạng thái được nén chặt, liên tục hoàn toàn về mặt vật lý, loại bỏ hiện tượng phân mảnh dữ liệu. Điều này giúp LSM-Tree trở nên vô đối khi so sánh với B+ Tree trong thao tác ghi dữ liệu; tuy nhiên vì phải tìm trên RAM, rồi dò qua nhiều file SSTable trên đĩa, LSM-Tree sẽ có điểm hạn chế khi thực hiện thao tác đọc dữ liệu, và thường phải sử dụng Bloom Filter để giúp quá trình đọc ổn định hơn. Ngoài ra, dù không gây phân mảnh dữ liệu nhưng cấu trúc này lại tiêu tốn thời gian để vi xử lý thực hiện các thao tác ngầm cho việc gộp file.

Do vậy, LSM-Tree không hẳn là vượt trội hoàn toàn so với B+ Tree mà điểm mạnh sẽ phát huy tùy thuộc vào mục đích sử dụng của hệ thống quản trị, cụ thể: B+ Tree vẫn sẽ phù hợp với các hệ thống ưu tiên việc đọc dữ liệu, hoặc cần các giao dịch nhất quán, và vẫn đang rất phổ biến trong các hệ quản trị cơ sở dữ liệu truyền thống như MySQL (InnoDB), PostgreSQL, SQL Server; trong khi đó LSM-Tree là cốt lõi của các hệ thống ghi dữ liệu khổng lồ, dữ liệu thời gian thực, hoặc các hệ thống NoSQL phân tán, tiêu biểu nhất là LevelDB do Google phát triển, RocksDB do Facebook phát triển, Apache Cassandra, và ScyllaDB.

7 Kết luận

Với vai trò là một sinh viên ngành công nghệ thông tin, việc tìm hiểu các cấu trúc dữ liệu có tính ứng dụng cao là một điều rất quan trọng. Trong khóa học lý thuyết Cấu trúc dữ liệu và giải thuật, chúng tôi đã được giới thiệu qua về cấu trúc B+ Tree nhưng vẫn chưa thực sự tìm hiểu sâu về nó. Và đồ án này đã giúp tôi hiện thực hóa cấu trúc dữ liệu trên bằng ngôn ngữ C++, đồng thời tạo cơ hội để tôi có thể tự tay chạy kiểm nghiệm và tìm hiểu nhiều kiến thức chuyên sâu hơn về kiến trúc phần cứng, phần mềm máy tính, các cấu trúc dữ liệu nâng cao, rất hữu ích cho công việc và nghiên cứu học thuật, đồng thời giúp tôi có cơ hội đóng vai như một tân kỹ sư hệ thống. Cụ thể, qua việc đối chiếu kết quả thực nghiệm giữa hai phương thức tìm kiếm là B-Tree và B+ Tree với các nền tảng lý thuyết cơ sở dữ liệu, tôi đã được làm sáng tỏ lý do vì sao B+ Tree được công nhận là tiêu chuẩn vàng cho các hệ quản trị cơ sở dữ liệu hiện đại.

Ngoài ra, thực hiện báo cáo này còn giúp tôi làm quen với kỹ năng nghiên cứu học thuật thông việc cài đặt mã nguồn để hoàn thiện ý tưởng, tích cực đọc các bài báo khoa học để có cái nhìn chuyên sâu hơn, từ đó vận dụng các kỹ năng như phân tích và tổng hợp dữ liệu, trình bày văn bản học thuật và kỹ năng tư duy khoa học để đưa ra bài báo cáo này một cách hoàn thiện và chính chu nhất.

Tài liệu

- [1] Feng Chen, David A. Koufaty, and Xiaodong Zhang. Understanding intrinsic characteristics and system implications of flash memory based solid state drives. In Proceedings of the eleventh international joint conference on Measurement and modeling of computer systems (SIGMETRICS '09), pages 181–192, New York, NY, USA, 2009. ACM.
- [2] Patrick O’Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O’Neil. The log-structured merge-tree (LSM-Tree). Acta Informatica, 33(4):351–385, 1996.
- [3] Jun Rao and Kenneth A. Ross. Making B+-Trees cache conscious in main memory. In Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, Dallas, TX, USA, 2000. ACM.
- [4] A. Yao. On random 2-3 trees. Acta Informatica, 9(2):159–170, 1978.